Empirical (Data-driven/ML) Modeling of Web Server

Out: *Mar 25, 2026*

Important Deadlines (in reverse chronological order) with ZERO MARGIN FOR EXTENSION. THESE ARE FINAL FINAL FINAL FINAL FINAL FINAL FINAL DEADLINES.

- *Final viva*: **Approximately around 7th-9th May.**
- *Final Submission (Report in the form of slides presentation, Code, link to data)* **:6th May**

- *Training data generation scripts and setup explanation report and possible demo*: **Anytime upto 24th April**
    - (Only script demo is required, data generation may not be completed, data generation is likely to take time. *This submission is optional for only those who want feedback and course-correction. Please note if you miss this opportunity, you will not get a chance for feedback - we'll just do final viva.)*

Assignment is to be done by TWO team members.

Overall Goal of the Assignment

In this assignment, you will try using 'data-driven' methods to model the performance of a **Web Server.** While the application remains the same as your Assignment 1 and 2, you will use a different php workload and deployment scenario to create somewhat not easily modelable behaviour which then justifies use of data-driven, empirical modelling methods rather than queuing models.

The overall goal is to (1) practically carry out one data driven modelling exercise and as a consequence 2) study whether indeed the data driven method worked better, or could you just as well have modelled the system analytically.

Precise Scenario to be Modelled

Since data-driven models make sense only when "simpler", computationally cheaper methods do not work, we have created a deployment where the Web server is deployed within a Docker environment. Since docker can add another 'layer' of resource limitations, this creates a complexity that may be difficult to be modelled analytically - there are a few more 'control knobs'. Furthermore, mechanisms that influence performance (memory management algorithms, I/O request serving algorithms) may be difficult to simulate (too many details, not

easy to learn) - so even simulation may not be the right approach. Thus there is enough motivation to try empirical methods for this purpose.

The Web Server Workload, Performance Metrics and Parameters

The Web server workload (php script which will be shared with you) is a workload with a mix of CPU and disk I/O. It can be called with the *loop count* and the *size of the IO* as *parameters.*

The *performance metric* of interest for the empirical model can be **response time**, as this is typically harder to predict. This is not to say that you should not model throughput - you can check if throughput is easier to predict using conventional methods as learnt in this course, if not, you can model throughput also.

The *parameters* of your "deployment" that should influence performance are many, and you should vary as many as you can to generate a meaningful data set that's worthy of being modelled using ML (empirical) methods.
Example parameters:

- Docker parameters
    - CPU fraction
    - Memory
    - Possible I/O rate limits
- Web workload parameters
    - Loop count
    - File size
- User parameters
    - Number of users, think time (if closed load) or arrival rate (if open load)
- Number of docker instances with the Web server running in each

The goal then is to model the response time, by using ML approaches, e.g. any of the approaches for *regression* (not *classification)* that form a part of the modern machine learning/artificial intelligence set of methods.

Specific Goal of the assignment

This assignment has a minimal goal as follows:

- Create an empirical model of steady state average response time of the web requests, as a function of various parameters.
- Demonstrate the efficacy of the model using data not used for training or testing (in the phase of fine tuning of model).
    - Use appropriate error quantification metrics. You can use the usual MAPE metric, but I strongly recommend using "90% Percentile of Absolute percent error".

- Some charts (4-5) that show measured Response time vs some parameter, while other parameters are kept constant - **must be plotted** and studied and 'sanity-checked' using methods we have learnt in class (validating asymptotes). In the same plots you could add a 'modelled' plot.
    - The rest of the graphs could be Predicted vs Measured scatter plots.

Note that this is a minimum goal, not a '10/10' goal.

The real task here is to use your imagination, your own curiosity and truly try to answer all kinds of questions around using these methods for performance modelling. E.g.

- What is the best method to generate training data ?
    - Explore methods that generate a good spread of configurations to be run for training data samples
- How to ensure the 'optimal' training data is generated - i.e. just enough required for a good model.
- What is the best model?
    - Explore more than one ML model: e.g. SVR, ANN etc
- How does this method compare with analytical or simulation modeling?

Further questions may come as you do the assignment and you should be curious and try to answer them.

Appendix: Instructions for running docker and the Web workload

- **Docker Installation:**
    1. Follow the instructions given in the following site to install docker. How to install docker.

    2. Also, make sure you add users in the Unix group so that you can run docker commands without superuser access. Run docker in non-root mode.

- **Contents in the starter code folder:**
    a. **server.php:** server script to be run inside a docker container
    b. **dockerStart.php:** It will create a container on the server machine
    c. **dockerStop.php**: It will stop and remove the container from the server machine
    d. **Dockerfile:** You need to build this to create an image of the docker container
    e. **clientScript.sh**: This script will run on a client machine. It will send requests to the server machine to create a container, run httperf, and finally destroy it.
    f. **Data.txt**: This file contains some data which will be written to some file inside the container. This can be changed.
- **How to set up:**

- Install docker on the server machine using the instructions provided above.
- After installation, check where the docker is installed using.

```
whereis docker
```

This command will tell you the location.
E.g.,**/snap/bin/docker or /usr/bin/docker**

And run

```
sudo -i
echo "www-data ALL=NOPASSWD: /snap/bin/docker" >> /etc/sudoers
```

This will ensure the docker containers start without a password through PHP scripts.

- Install the apache server on the server machine.
- Change the port of the apache server to 8080. Follow the link to know how to do it.
- Create a folder on the server machine and put the Dockerfile, data.txt, and server.php files in it.
- Open a terminal. Go to the folder and run the command:

```
docker build -t server_image:1.0 .
```

- Put dockerStart.php and dockerStop.php inside the /var/www/html folder.
- Put the clientScript.sh on the client machine.
- Also, install the screen utility. This will help you run your load-testing script in the background. Link to man page.

- **Instructions to load test:**
  a. For load testing, you must run the clientScript on the client machine.
  b. Ensure that the IP and port in line 14 and 18 in clientScript.sh is configured correctly according to your server machine. (This script is just for you to understand how to run the scripts for starting and stopping the docker container. You are encouraged to write your own scripts).
  c. There are so many parameters you can vary in this assignment; we primarily want you to vary device_write, memory, a fraction of docker container CPUs, and the size the server script will write in the file and cpuLoad parameter.

d. Experiment with these parameters, try to saturate the server, and get the results in terms of response time.
e. Use the ***dockerstat*** command to get metrics of docker containers.

- **Some precautions to be taken:**
  a. If the load test is stopped in between, please make sure you remove the container from the server machine using the following command.

```
docker stop <container_name>
docker rm <container_name>
```

  b. By default, the disk memory allocated to each docker container is 10GB. So ensure that in one httperf run, the disk storage does not exhaust by writing too much data to disk.